

Recrearía solamente la tabla suplementaria, **no la cohorte ni 23B**:

DROP TABLE IF EXISTS

bu_frd.model_party_utilization_monthly_2022_2025;

CREATE TABLE

bu_frd.model_party_utilization_monthly_2022_2025

USING PARQUET

AS

SELECT

party_id,
observation_date,
feature_month,
feature_window,

/* =====
Structurally eligible accounts
===== */

COUNT(
DISTINCT CASE
WHEN active_account_flag = 1
AND UPPER(TRIM(CAST(porf_typ_cde AS STRING)))
IN ('C','R')
AND cr_lmt_amt > 0
THEN acct_id
END
) AS utilization_eligible_accounts,

/* =====
Accounts with an observed balance
===== */

COUNT(
DISTINCT CASE
WHEN active_account_flag = 1
AND UPPER(TRIM(CAST(porf_typ_cde AS STRING)))
IN ('C','R')
AND cr_lmt_amt > 0
AND curr_bal_amt IS NOT NULL
THEN acct_id
END
) AS utilization_observed_accounts,

```

/* =====
Balance / limit only where both
components needed for utilization exist
===== */

```

```

SUM(
  CASE
    WHEN active_account_flag = 1
      AND UPPER(TRIM(CAST(porf_typ_cde AS STRING)))
        IN ('C','R')
      AND cr_lmt_amt > 0
      AND curr_bal_amt IS NOT NULL
      THEN curr_bal_amt
    END
) AS utilization_balance_sum,

```

```

SUM(
  CASE
    WHEN active_account_flag = 1
      AND UPPER(TRIM(CAST(porf_typ_cde AS STRING)))
        IN ('C','R')
      AND cr_lmt_amt > 0
      AND curr_bal_amt IS NOT NULL
      THEN cr_lmt_amt
    END
) AS utilization_limit_sum,

```

```

/* =====
Average account-level utilization
===== */

```

```

AVG(
  CASE
    WHEN active_account_flag = 1
      AND UPPER(TRIM(CAST(porf_typ_cde AS STRING)))
        IN ('C','R')
      AND cr_lmt_amt > 0
      AND curr_bal_amt IS NOT NULL
      THEN
        1.0 * curr_bal_amt / cr_lmt_amt
    END
) AS avg_account_utilization,

```

```

/* =====
Portfolio-weighted utilization
===== */

```

```

CASE
  WHEN SUM(
    CASE
      WHEN active_account_flag = 1
      AND UPPER(TRIM(CAST(porf_typ_cde AS STRING)))
        IN ('C','R')
      AND cr_lmt_amt > 0
      AND curr_bal_amt IS NOT NULL
      THEN cr_lmt_amt
    END
  ) > 0

  THEN
    1.0 *
    SUM(
      CASE
        WHEN active_account_flag = 1
        AND UPPER(TRIM(CAST(porf_typ_cde AS STRING)))
          IN ('C','R')
        AND cr_lmt_amt > 0
        AND curr_bal_amt IS NOT NULL
        THEN curr_bal_amt
      END
    )
    /
    SUM(
      CASE
        WHEN active_account_flag = 1
        AND UPPER(TRIM(CAST(porf_typ_cde AS STRING)))
          IN ('C','R')
        AND cr_lmt_amt > 0
        AND curr_bal_amt IS NOT NULL
        THEN cr_lmt_amt
      END
    )

  ELSE NULL
END AS weighted_utilization,

```

```

MAX(
  CASE
    WHEN active_account_flag = 1
    AND UPPER(TRIM(CAST(porf_typ_cde AS STRING)))
      IN ('C','R')
    AND cr_lmt_amt > 0
    AND curr_bal_amt IS NOT NULL

```

```

        THEN
            1.0 * curr_bal_amt / cr_lmt_amt
        END
    ) AS max_account_utilization

FROM
    bu_frd.model_trades_transition_13m_2022_2025

WHERE
    feature_window IN ('PRIOR_9M','RECENT_3M')

GROUP BY
    party_id,
    observation_date,
    feature_month,
    feature_window;

```

Después validaría directamente que average y weighted tengan la misma condición de disponibilidad:

```

SELECT

    COUNT(*) AS party_months,

    SUM(
        CASE
            WHEN avg_account_utilization IS NOT NULL
            THEN 1 ELSE 0
        END
    ) AS avg_util_available,

    SUM(
        CASE
            WHEN weighted_utilization IS NOT NULL
            THEN 1 ELSE 0
        END
    ) AS weighted_util_available,

    SUM(
        CASE
            WHEN
                (avg_account_utilization IS NULL
                 AND weighted_utilization IS NOT NULL)

            OR

                (avg_account_utilization IS NOT NULL
                 AND weighted_utilization IS NULL)

```

```
        THEN 1 ELSE 0
    END
) AS availability_mismatch
```

```
FROM
    bu_frd.model_party_utilization_monthly_2022_2025;
```

Aquí quiero:

```
party_months      = 8,263,068
availability_mismatch = 0
```

También podemos comprobar que nuestro cálculo original de average utilization no cambió:

```
SELECT

    COUNT(*) AS rows_compared,

    SUM(
        CASE
            WHEN
                b.avg_utilization IS NULL
                AND u.avg_account_utilization IS NULL
            THEN 0

            WHEN
                b.avg_utilization IS NOT NULL
                AND u.avg_account_utilization IS NOT NULL
                AND ABS(
                    b.avg_utilization
                    - u.avg_account_utilization
                ) < 0.0000001
            THEN 0

            ELSE 1
        END
    ) AS avg_utilization_mismatch

FROM
    bu_frd.model_party_behavior_monthly_2022_2025 b

LEFT JOIN
    bu_frd.model_party_utilization_monthly_2022_2025 u

    ON b.party_id = u.party_id
    AND b.observation_date = u.observation_date
    AND b.feature_month = u.feature_month;
```

Esperamos:

avg_utilization_mismatch = 0

Eso nos confirma que la corrección solamente afecta la definición de weighted utilization.

Después debemos regenerar 23C y 23D

No hay que rehacer:

cohort
controls
13-month keys
98M trades
transitions
party-month 23B

Todo eso permanece congelado.

Solo:

corrected weighted utilization

↓

rerun 23C windows

↓

rerun 23D current features

Y volver a comprobar:

```
SELECT
  COUNT(*) AS rows,
  COUNT(DISTINCT party_id) AS parties,

  SUM(
    CASE
      WHEN utilization_change_vs_baseline IS NULL
      THEN 1 ELSE 0
    END
  ) AS utilization_change_missing,

  SUM(
    CASE
      WHEN weighted_utilization_change_vs_baseline IS NULL
      THEN 1 ELSE 0
    END
  ) AS weighted_util_change_missing
```

FROM

bu_frd.model_behavior_features_current_2022_2025;

Con la definición corregida, esperaríamos que los dos missing counts sean iguales o, como mínimo, que cualquier diferencia tenga una explicación explícita. Con las mismas cuentas observadas, deberían coincidir.

Luego sí cerramos 23E

La decisión que ya podemos congelar para Z-score es:

Z SCORE CHALLENGERS

- ✓ active_accounts
- ✓ balance
- ✓ avg_utilization
- ✓ weighted_utilization

NO Z SCORE

- ✗ past_due
- ✗ delinquency_share
- ✗ deterioration
- ✗ adverse

Y hay otra decisión importante: para **active_accounts**, el hecho de que **SD=0** no es un dato faltante. Significa que el número de cuentas activas fue **estable durante los 9 meses**. Por eso tendremos por separado:

active_accounts_z
active_accounts_z_available_flag
active_accounts_prior_sd_zero_flag

Lo mismo conceptualmente para balance/utilization cuando corresponda.

Una vez confirmado el **availability_mismatch = 0**, podemos construir el **23E definitivo** con esos cuatro Z-scores y pasar inmediatamente al Paso 24, donde comenzaremos a comparar las señales en TRAIN antes de entrenar los modelos.